

IE 300 Case Study 2: Predicting Flight Delays

1 Objectives

The purpose of this study is to develop a very popular predictive model called Naive-Bayes. You will use Python as a tool to analyze data of flight delays in order to determine the likelihood of delays. Through this case study, you will learn how to:

1. Read and analyze data using Python.
2. Apply the concepts of conditional probability and Bayes' theorem.
3. Develop a simple Naive-Bayes classifier model with m -estimates.

2 Predicting Delays

As any frequent flier knows, flight delays are an inevitable part of commercial air travel. In the previous case study, you were able to determine which airline is the fastest for a particular route. In doing so, you had to take into account departure and arrival delays of each airline. While the cause for delays are largely unpredictable, we can get some insights on which flights are more likely to be delayed without taking weather conditions into consideration.

An obvious predictor of delays is the airline itself. For example, in 2014, only 74.34% of American Airline's flights were on-time while Delta had a 85.21% on-time record [1]. How can we use historical data to make predictions about delays? You may have already learned about several predictive models such as ordinary least-squares. We will study a very popular model in statistics and machine learning called Naive-Bayes.

2.1 Naive Bayes Classifier

Recall that Bayes' theorem tells you the probability of an event based on conditions possibly related to that event. That is, given events A and B , we have

$$P(A | B) = \frac{P(A)P(B | A)}{P(B)}. \quad (1)$$

The Naive-Bayes classifier is based on Bayes' theorem with the assumption of independence between attributes. Suppose we have a vector $X = (x_1, x_2, \dots, x_n)$ representing n **attributes** (independent variables). The Naive-Bayes' classifier helps us compute $P(C_k | X)$, where $C_1, C_2, \dots$, represent possible outcomes (**classes**).

If we are trying to decide whether a particular flight is likely to be delayed, the outcome would be from the set $\{Y, N\}$, corresponding to the flight being delayed (Y) or not (N). The vector X would contain information about the flight (e.g., airline, origin airport, destination airport) that could give us some insight into which class (Y or N) will occur. One approach to predicting the

class is to pick the value C_k that is most probable, given that we know the values of the attributes. In other words, we pick the C_k that maximizes $P(C_k | X)$. Since the vector X is known, and $P(C_k | X) = P(C_k \cap X) / P(X)$ by the definition of conditional probability, we get that $P(X)$ will be the same for all possible classes, and we can instead focus on selecting the class that maximizes $P(C_k \cap X)$. We call $P(C_k \cap X)$ the **score** for class C_k , and we call $P(C_k | X)$ the **posterior probability** for class C_k .

Noting that the attribute vector X typically contains values of several attributes, we can rewrite $P(C_k \cap X)$ as $P(C_k \cap x_1 \cap x_2 \cap \dots \cap x_n)$, where the event x_i tells us the value of attribute i . In practice, the symbol $\cap$ is often replaced with a comma when intersecting many events. By repeatedly applying the definition of conditional probability, we get

$$\begin{aligned} P(C_k \cap X) &= P(C_k) P(X | C_k) \\ &= P(C_k) P(x_1 \cap \dots \cap x_n | C_k) \\ &= P(C_k) P(x_1 | C_k) P(x_2 \cap \dots \cap x_n | C_k) \\ &\quad \vdots \\ &= P(C_k) P(x_1 | C_k) P(x_2 | C_k \cap x_1) \dots P(x_n | C_k \cap x_1 \cap \dots \cap x_{n-1}). \end{aligned} \quad (2)$$

This decomposes $P(C_k \cap X)$ into a product of the probability of outcome C_k and conditional probabilities of each attribute. We assume conditional independence between the attributes (hence, the “naive” portion of the name). That is,

$$P\left(x_i \mid C_k \cap \left(\bigcap_{j \neq i} x_j\right)\right) = P(x_i | C_k) \quad (3)$$

for all $i \in \{1, 2, \dots, n\}$. This allows us to discard all conditions except C_k in equation (2). Hence, the equation simplifies to

$$P(C_k \cap X) = P(C_k) \prod_{i=1}^n P(x_i | C_k). \quad (4)$$

With this definition, we can define a **classifier** (a decision rule). A common classifier is to pick the most probable class, as discussed above. This is called the **maximum a posterior** (MAP) rule, which assigns the label

$$\operatorname{argmax}_k P(C_k) \prod_{i=1}^n P(x_i | C_k). \quad (5)$$

2.2 m -Estimates

The conditional densities $P(x_i | C_k)$ can be defined in different ways (e.g., Normal, Poisson) depending on the problem. If our dataset is small, we often do not know the underlying distributions. When the attribute x_i is discrete, an intuitive way to estimate $P(x_i | C_k)$ is to define

$$P(x_i | C_k) = \frac{\hat{n}}{n}. \quad (6)$$

Here $\hat{n}$ is the number of observations where the class is C_k and the i^{th} attribute is x_i , and n is the number of observations where the class is C_k . However, for small datasets, we may find that $\hat{n} = 0$ or $n = 0$, which is a problem. A way to avoid this problem is to use m -estimates.

$$P(x_i | C_k) = \frac{\hat{n} + mp}{n + m}. \quad (7)$$

Here m is called the *equivalent sample size* and p is the *prior estimate* of $P(x_i | C_k)$. A typical choice for p is $1/(\text{no. of possible values for attribute } i)$, which assumes that all attribute values are equally likely. The idea behind m -estimates is to pretend we have m extra observations with class C_k , with $m \cdot p$ of them having the i^{th} attribute x_i . Essentially, m says how confident we are of our prior estimate p , since as m increases, we have $P(x_i | C_k) \rightarrow p$.

Thus, we can compute $P(x_i | C_k)$ for each attribute x_i , and then we can compute the score of class C_k using equation (4).

2.3 Posterior Probabilities

To compute the posterior probability $P(C_k | X)$, we use the fact that posterior probabilities of all classes must sum to 1, i.e.,

$$\sum_k P(C_k | X) = 1. \quad (8)$$

Since $P(C_k | X) = P(C_k \cap X) / P(X)$ for each class C_k , we get

$$P(X) = \sum_k P(C_k \cap X). \quad (9)$$

Hence, one can compute posterior probabilities as follows:

1. For each class C_k , compute the score $P(C_k \cap X)$.
2. Sum the scores of all classes to obtain $P(X)$.
3. Divide the score of each class C_k by $P(X)$ to obtain the posterior probability $P(C_k | X)$.

3 Example 1

Consider the following table (also available as `flight-delay.example.csv`) containing information of delays.

#	Origin	Destination	Airline	Delay
1	DFW	ORD	Delta	Y
2	DFW	ORD	Delta	N
3	DFW	ORD	Delta	Y
4	LAX	ORD	Delta	N
5	LAX	ORD	United	Y
6	LAX	LGA	United	N
7	LAX	LGA	United	Y
8	LAX	LGA	Delta	N
9	DFW	LGA	United	N
10	DFW	ORD	United	Y

The class is the variable Delay and the attributes are Origin, Destination, and Airline. Classify DFW-LGA on Delta using Naive-Bayes with MAP and m -estimates. Assume $m = 3$.

First, note that DFW-LGA on Delta is not contained in the table. However, we can compute the delay scores for this flight using equation (4). Using an equivalent sample size of $m = 3$, we compute the relevant conditional probabilities as follows:

$$P(\text{DFW} | Y) = \frac{3 + 3/2}{5 + 3} = \frac{9}{16} = 0.5625 \quad P(\text{DFW} | N) = \frac{2 + 3/2}{5 + 3} = \frac{7}{16} = 0.4375$$

$$\begin{aligned} P(\text{LGA} \mid Y) &= \frac{1 + 3/2}{5 + 3} = \frac{5}{16} = 0.3125 & P(\text{LGA} \mid N) &= \frac{3 + 3/2}{5 + 3} = \frac{9}{16} = 0.5625 \\ P(\text{Delta} \mid Y) &= \frac{2 + 3/2}{5 + 3} = \frac{7}{16} = 0.4375 & P(\text{Delta} \mid N) &= \frac{3 + 3/2}{5 + 3} = \frac{9}{16} = 0.5625 \end{aligned}$$

For $P(\text{DFW} \mid Y)$, we have 5 observations where Delay = Y , of which three observations have $x_i = \text{DFW}$. Thus, $n = 5$ and $\hat{n} = 3$. Origin takes two possible values (DFW and LAX), so $p = 1/2$. Since each of Y and N occur in 5 of the 10 observations, we have $P(Y) = P(N) = 0.5$. On applying equation (4), we get the scores of Y and N :

$$\begin{aligned} P(Y \cap (\text{DFW}, \text{LGA}, \text{Delta})) &= P(Y) P(\text{DFW} \mid Y) P(\text{LGA} \mid Y) P(\text{Delta} \mid Y) \\ &= \frac{1}{2} \cdot \frac{9}{16} \cdot \frac{5}{16} \cdot \frac{7}{16} = \frac{315}{8192} \approx 0.0385. \\ P(N \cap (\text{DFW}, \text{LGA}, \text{Delta})) &= P(N) P(\text{DFW} \mid N) P(\text{LGA} \mid N) P(\text{Delta} \mid N) \\ &= \frac{1}{2} \cdot \frac{7}{16} \cdot \frac{9}{16} \cdot \frac{9}{16} = \frac{567}{8192} \approx 0.0692. \end{aligned}$$

Since N has a larger score, we classify DFW-LGA on Delta as N , i.e., we predict that the flight from DFW to LGA on Delta is likely to not be delayed.

Moreover, the posterior probabilities are:

$$\begin{aligned} P(Y \mid \text{DFW}, \text{LGA}, \text{Delta}) &= \frac{315}{315 + 567} = \frac{5}{14} \approx 0.357, \\ P(N \mid \text{DFW}, \text{LGA}, \text{Delta}) &= \frac{567}{315 + 567} = \frac{9}{14} \approx 0.643. \end{aligned}$$

4 Exercise 1

Using the following table (also available as `flight-delay.small.csv`), will the flight SEA-ATL on Southwest with good weather be delayed? Use m -estimates for the conditional probabilities with $m = 4$.

#	Origin	Destination	Airline	Weather	Delay
1	SEA	ATL	WN	Poor	N
2	SEA	BOS	AA	Good	Y
3	SEA	ATL	UN	Poor	Y
4	SFO	BOS	WN	Poor	Y
5	SFO	ATL	AA	Good	N
6	SFO	BOS	UN	Poor	Y
7	SFO	BOS	UN	Good	N
8	SEA	BOS	WN	Poor	Y

5 Real World Data

Let us now use real historical flight data to predict flight delays. The dataset `flight-delay.large.csv` contains 2,346 observations of flights departing from JFK or SFO to ATL, LAS or ORD in November 2015. There are 5 variables.

Variable Name	Description
Carrier	IATA Carrier identification
Origin	Origin airport
Destination	Destination airport
Depature Delay	Difference between scheduled and actual departure time (minutes)
Arrival Delay	Difference between scheduled and actual arrival time (minutes)

5.1 Exercise 2

Using `flight-delay.large.csv`, write Python code to perform the following:

- Read the dataset.
- Compute the total delay time for each flight. The total delay time is the sum of departure delay and arrival delay time.
- Assign any flight with total delay time greater than 15 minutes with *Y* for being delayed. Otherwise, the flight is not delayed and is assigned *N*.

5.2 Exercise 3

Using your Python code from the previous exercise, for each of the following flights, compute the probabilities of delay and not delay given the route and carrier and then classify whether the flight is delayed using Naive-Bayes (assume $m = 3$).

- JFK-LAS on American Airlines (AA).
- JFK-LAS on JetBlue (B6).
- SFO-ORD on Virgin Airlines (VX).
- SFO-ORD on Southwest Airlines (WN).

Specifically, your output should contain:

- $P(\text{Delay} \mid \text{Origin, Destination, Carrier})$.
- $P(\text{Not delay} \mid \text{Origin, Destination, Carrier})$.
- Classification of whether the flight is delayed or not.

References

- [1] Skift. The world's best and worst on-time airlines, Nov 2014. URL: <https://skift.com/2014/11/18/the-worlds-best-and-worst-on-time-airlines/>.

A Grading Rubric (40 points)

For programming problems, students must submit both the code and the output.

A.1 Exercise 1 (11 points)

1. For each label (Y or N), showing your work and using the correct formula gives 5 points (1 for each multiplicative term).
2. Showing how to use the scores (or probabilities) to obtain the correct prediction gives 1 point.

A.2 Exercise 2 (3 points)

1. Read the dataset: 1 point
2. Compute the total delay time: 1 point
3. Assign Y or N label: 1 point

A.3 Exercise 3 (26 points)

The official solution uses functions to avoid repetition of code. (E.g., it has a function called ‘predict’, which takes a flight’s details (source, destination, airline) as input and displays the posterior probabilities and the prediction. This avoids repeating the code for each of the 4 inputs (a) to (d).)

The rubric below is written with the official solution in mind, but students are free to follow a different outline. Specifically, there is no penalty for code repetition; points will simply be split across each repetition.

1. 11 points for code that computes appropriate aggregations:
 - (a) 2 points to compute aggregations needed for n .
 - (b) 6 points to compute aggregations needed for $\hat{n}$.
 - (c) 3 points to compute aggregations needed for p .
2. 9 points for code that uses aggregates to compute scores:
 - (a) 2 points for the $P(C_k)$ term.
 - (b) 4 points for the correct numerator of the $P(x_i | C_k)$ term.
 - (c) 2 points for the correct denominator of $P(x_i | C_k)$ term.
 - (d) 1 point to combine these terms to get the scores for each label.
3. 2 points for code that uses the scores to compute posterior probabilities.
4. 4 points for outputs. For each of the 4 inputs (a) to (d):
 - (a) 0.5 point for the correct posterior probabilities of Y and N.
 - (b) 0.5 point for the correct prediction (Delayed or Not delayed).

A.4 Extra credit

1. 2 points for not hard-coding the number and names of input columns. This makes it easy to adapt the solution to different datasets.
2. For datasets with a very large number of rows, most operations take a very long time. Avoiding needless repetition can considerably speed up computation. 2 points will be awarded to code that is close to optimal in terms of number of passes over data.